Project Title

SummarIQ – AI-Powered PDF Summarizer Application for Scientific Papers

Project Overview

Objective

The primary goal of this project is to develop a simplified AI system that summarizes research papers in PDF format. The system will extract main points, including technical content like equations, tables, and key findings. For the first semester, the focus will be on text extraction and generating concise summaries using pretrained NLP models.

Scope

In Scope (Semester 1):

- Upload a PDF research paper and extract text using Python libraries.
- Identify and retain key technical content, including equations.
- Generate extractive or abstractive summaries using pretrained NLP models.
- Display the summarized text in a clean interface.
- iOS demo app: Simple interface to upload PDF, process, and show summary .

Out of Scope (Semester 1):

- Handling images, graphs, or charts in summaries.
- Deep understanding of equations (semantic evaluation).
- Multi-document summarization or literature survey generation.
- Real-time summarization or cloud deployment.

Limitations:

- Summaries may omit fine details from complex equations.
- Works best on text-heavy PDFs.
- Dependent on quality of PDF formatting.

AI Techniques and Tools

Core AI Methods (Semester 1):

- **Text Extraction:** GROBID, *PyPDF2, pdfplumber* for PDF → text conversion.
- **Preprocessing:** Remove headers, footers, page numbers; normalize text.
- **Summarization Models:**
    - Pretrained Transformers: *T5-small, BART* for abstractive summaries.
    - Extractive summarization: *LexRank* or *sumy* for selecting key sentences.

Programming Libraries:

- `transformers` → NLP models
- `PyPDF2`, `pdfplumber` → PDF text extraction
- `NumPy`, `Pandas` → data processing
- `SwiftUI` → iOS interface
- `Flask` → optional backend API connecting Python model to iOS app

Semester Milestone:

- Extract text from research paper PDFs.
- Identify sections and equations (as LaTeX strings or placeholders).
- Generate concise summaries (3–5 sentences).
- Display summary in an iOS app interface (upload → processing → result).
- Stretch goal: Highlight equations or key points in the summary.

Future Scope:

- **Equation Awareness:** Integrate tools to parse and semantically understand equations.
- **Multi-Document Summarization:** Summarize multiple papers into one literature review.
- **Graph & Table Summarization:** Include tables, charts, and images in summaries.
- **Deep Learning Enhancements:** Fine-tune transformer models on research paper datasets.
- **iOS Deployment:** Convert model to Core ML for offline summarization on iPhone.

Data Plan:

- **Custom Data:** Select 10–20 PDFs from arXiv or other open-access research papers for testing/demo.
- **Public Datasets:**
    - **ScisummNet** – NLP summaries of scientific papers.
    - **SurveySum** – Summaries of multiple scientific papers.
    - **MS2** – Large-scale dataset with documents and summaries.
    - **cPAPERS** – Papers containing equations and tables for summarization.

Stakeholders:

Project Team:

- Satyabrata Das – AI Developer & iOS Developer → Implement AI pipeline, app interface, and integration.

End Users:

- Researchers & students → Quickly read research papers.

- Educators → Summarize content for lectures.

Other Stakeholders:

- Universities → Can use lightweight summarization tools.
- Open-access repositories → Provide research PDFs.

Self-Rating of Confidence (Beginner Stage):

- Transformers (Hugging Face): 3/10 → New to NLP fine-tuning.
- PDF Processing (PyPDF2, pdfplumber): 7/10 → Comfortable extracting text.
- iOS/SwiftUI: 6/10 → Able to create simple app interfaces and integrate backend.

Computing Infrastructure:

Objective and Scope

The primary goal of this infrastructure is to support a pipeline for extracting text from research paper PDFs, preserving mathematical expressions (as LaTeX or [Equation] tokens), and generating concise 3–5 sentence summaries using abstractive or extractive methods. The infrastructure must enable fast, cost-effective, and reproducible experimentation and deployment across development, training, and demo phases.

System Objectives and Functional Tasks

Core Tasks

1. **Text Extraction and Cleanup** – Extract textual content from PDFs, removing headers, footers, and pagination using `pdfplumber` or `PyPDF2`.
2. **Equation Detection and Preservation** – Identify inline and block-level math content; store them as LaTeX strings or `[Equation]` placeholders.
3. **Summarization** –
    - **Abstractive**: Use `T5-small` or `BART-base` fine-tuned on scientific text.
    - **Extractive fallback**: Apply `LexRank` when abstractive models underperform.
4. **iOS Demo Application** – Enable file upload, cloud-based text processing, and display of summaries with equation highlights.

Data Types

- **Primary**: Plain text from PDFs, including LaTeX math tokens.
- **Secondary**: Metadata (title, author list, abstract) and text-based tables.
- **Out of Scope (Semester 1)**: Figures, semantic equation parsing, and image extraction.

Quantitative Performance Benchmarks

Metric	Target	Justification / Notes
Latency (Inference)	≤ 2 sec (GPU), ≤ 10 sec (CPU)	Ensures interactivity in demo settings using T5-small or BART-base.
Throughput (Server)	5–20 requests/minute per GPU	Supports small concurrent user base; scalable horizontally.
Summarization Quality	ROUGE-1: 25–40	Realistic for fine-tuned small transformer models on limited data.
Accuracy Validation	≥ 80% human evaluator pass rate	Ensures summaries capture main paper content.
Cost (Training + Inference)	≤ $50/month (demo phase)	Achievable using spot GPU instances (A10/T4).

Performance Validation

- Evaluate summary quality using **ROUGE-1/2/L** metrics.
- Measure **p50/p95 latency** using API logs during test runs.
- Conduct **load testing** with tools like *Locust* or *JMeter* to simulate concurrent requests and measure auto-scaling thresholds.

Hardware and Resource Planning:

Component	Minimum Spec (Development)	Recommended Spec (Training)	Justification
CPU	8 vCPUs	16 vCPUs	For parallel data processing during training.
RAM	32 GB	64 GB	Required for PDF parsing and model fine-tuning.
GPU (VRAM)	12–16 GB (T5-small/BART-base)	40+ GB (BART-large)	Ensures smooth fine-tuning and inference.
Storage	200 GB SSD	1 TB SSD (if caching datasets)	For datasets, model checkpoints, and logs.

Example GPU Options

- **Local:** NVIDIA RTX 3090 / 4090
- **Cloud:** NVIDIA A10, T4, A100 instances on AWS/GCP

Decision:
A single GPU node (T4/A10 class) will serve both training (spot/preemptible instances) and inference (on-demand, single-GPU endpoint). Larger models (e.g., BART-large) may use A100-class hardware if budget allows.

Software Environment:

Layer	Tools / Frameworks	Purpose
OS	macOS / Ubuntu	Standard ML development environment
Frameworks	PyTorch + Hugging Face Transformers	Core for T5/BART fine-tuning
PDF Processing	pdfplumber, PyPDF2	Text and math extraction
NLP Preprocessing	regex, NLTK, spaCy	Tokenization, sentence segmentation
Summarization Models	T5-small, BART-base, LexRank	Abstractive and extractive summarization
Backend	Flask / FastAPI + gunicorn + uvicorn	API inference server
Containerization	Docker	Reproducibility and deployment consistency
Frontend	SwiftUI (iOS)	User-facing upload and summary display

Deployment Plan:

Phase	Platform	Description
Development	Local GPU / VM	Training small models and pipeline validation.
Semester 1 (Local Host)	**Local Server / Flask**	Initial deployment and testing of the Fast API locally before cloud deployment.
Demo Deployment	Cloud GPU instance (AWS/GCP)	Fast API with GPU-backed inference endpoint.
Storage	S3 / GCS	PDF uploads and processed summaries.
Database	PostgreSQL / Firebase	User metadata and summary caching.
Future Expansion	Core ML conversion	Enable on-device inference in later semesters.

Tradeoff Analysis:

Factor	Option A (T5-small)	Option B (BART-base)	Tradeoff Summary
Model Size	60M params	140M params	BART-base yields

			higher quality but higher cost/latency.
Inference Latency (GPU)	~1.5 sec	~3.5 sec	T5-small preferred for interactive demo.
GPU Memory Requirement	~12 GB	~24 GB	BART-base may require a higher-tier GPU.
Monthly Cloud Cost	~$25	~$60	Cost scales with GPU class and runtime.
ROUGE-1 Score (Expected)	30–35	35–45	Slightly higher accuracy at higher resource cost.

Decision:
For Semester 1, T5-small is selected for its balance between cost, latency, and quality. BART-base will be considered for later improvements if resource constraints allow.

Scalability and Load Validation:

Scaling Strategy

- Horizontal scaling: Multiple Flask/FastAPI replicas behind a load balancer.
- Auto-scaling policy: Trigger new instances when GPU utilization >70% or latency >3 sec (p95).
- Batching and caching: Use cached summaries for repeated PDFs; batch small requests to improve GPU utilization.

Load Testing

- Use Locust or JMeter to simulate concurrent users and validate scalability.
- Measure performance under 5, 10, and 20 concurrent requests to confirm that average latency remains under 2 seconds for GPU-backed inference.

Risk Management and Triggers:

Risk	Trigger Condition	Mitigation Strategy
Latency Degradation	Median latency > 2 sec or p95 > 5 sec	Apply quantization (8-bit), enable mixed precision, or upgrade GPU instances.
Budget Overrun	Monthly cloud spend > 25% of plan	Switch to preemptible instances or reduce model complexity.
Low ROUGE / Quality	ROUGE-1 < 25 or <70% human pass rate	Fine-tune longer or apply domain-specific data augmentation.
Scalability Limit	>20 concurrent users degrade performance	Enable horizontal auto-scaling and response caching.

Security, Privacy, and Ethics (Trustworthiness):

Trustworthiness is embedded throughout the AI lifecycle, ensuring that the summarization model remains **fair, transparent, privacy-preserving, and secure**. Each stage of the system's lifecycle — from problem definition to post-deployment — explicitly considers ethical and technical safeguards.

Problem Definition

Strategy: Identify ethical and fairness risks before system design begins.
Plan: Examine possible harms such as biased summaries, factual distortion, or the omission of critical details. Assess how these risks may affect users like students, researchers, or publishers.
Tool/Method: Apply the **Data Ethics Canvas** to document potential risks, stakeholder concerns, and mitigation actions.
Measurable Criteria: Completion of an ethical-risk checklist prior to data collection.

Data Collection

Strategy: Use representative, unbiased, and privacy-conscious data sources.
Plan: Collect papers from **arXiv** and similar open repositories, remove sensitive metadata (e.g., author names), and balance datasets across domains.
Technical Implementation:

- **Snorkel**: Generate synthetic data to reduce underrepresentation of specific paper types.
- **IBM Diffprivlib**: Apply differential privacy to protect metadata.
  Metrics: Monitor dataset balance using **domain representation ratios** and **KL-divergence**.

Model Development

Strategy: Build fair, explainable, and robust summarization models.
Plan: Fine-tune **BART** models while assessing fairness across scientific domains.
Technical Implementation:

- **Fairlearn**: Compare summarization quality (ROUGE, F1) across disciplines such as physics, medicine, and computer science.
- **SHAP**: Explain model attention and detect systematic bias in how sentences are weighted.
  Audit Criteria: Retraining is triggered if the fairness gap (e.g., ROUGE difference across domains) exceeds **10%**.

Deployment

Strategy: Deploy the summarization system securely and transparently.
Plan: Package the model using **Docker** and serve via **FastAPI** with **JWT-based authentication** and HTTPS. Log API activity for traceability.

Tool: BentoML for serving, versioning, and monitoring.
User Transparency:

- Provide a **Model Card** describing data sources, limitations, and update frequency.
- Display a short disclaimer such as: *"Summaries may omit technical nuances — please verify with the original paper."*

Monitoring and Maintenance

Strategy: Continuously evaluate fairness, drift, and performance post-deployment.
Plan:

- Use **NannyML** to detect data or concept drift monthly.
- Run quarterly fairness audits with **Fairlearn**.
- Gather user feedback through rating surveys to guide retraining.

Metrics Tracked:

- **Quality:** ROUGE-L, factual consistency (FactCC).
- **Fairness:** Inter-domain performance differences, sentiment balance.
- **Drift:** Population Stability Index (PSI).
  Retraining Policy: Retrain when PSI > 0.05 or quality metrics fall by more than 5%.
  Transparency: Publish changelogs and performance updates after each model revision.

Human–Computer Interaction (HCI)

For this project, HCI principles are applied throughout the design process to ensure the **LaTeX summarization system** is simple, accessible, and valuable for students, researchers, and educators. User needs and feedback drive every stage — from requirement gathering to post-deployment evaluation.

Understand User Requirements

Approach:

- Conducted short Google Form surveys with students and faculty to understand how they currently read and summarize LaTeX-based research papers.
- Followed up with semi-structured interviews (3–5 students, 1–2 faculty members) to explore pain points and expectations.

Insights Expected:

- Users want faster ways to skim LaTeX papers and extract main ideas.
- Equations and mathematical notations must be preserved accurately in summaries.
- Users prefer a lightweight, mobile-friendly interface for easy access.

Tool:
Used **Miro Affinity Diagramming** to group feedback into categories such as *"time-saving," "clarity of equations,"* and *"ease of reading."*

Create Personas and Scenarios

Personas

1. **Student Persona – "Riya"**
   Profile: 20-year-old undergraduate computer science student.
   Motivation: Needs short, clear summaries for assignments and LaTeX-based project papers.
   Pain Points: Reading long `.tex` documents with equations is tedious on small screens; I want quick, structured insights.
   Context of Use: Uses the mobile app between classes or during study sessions.
2. **Researcher Persona – "Arjun"**
   Profile: 26-year-old PhD student in Artificial Intelligence.
   Motivation: Needs precise summaries that retain equations, LaTeX syntax, and domain-specific content.
   Pain Points: Most existing summarizers fail to handle LaTeX markup or skip equations entirely.
   Context of Use: Uses a desktop/laptop while analyzing technical LaTeX manuscripts.
3. **Educator Persona – "Dr. Meera"**
   Profile: 40-year-old computer science lecturer.
   Motivation: Prepares lecture materials from LaTeX-based research papers.
   Pain Points: Reviewing multiple `.tex` files before class preparation is time-consuming.
   Context of Use: Uses a web or desktop interface during lecture preparation or research review sessions.

Scenarios

- **Scenario 1:** Riya uploads a `.tex` file, and the system generates a 3–5 sentence summary readable in under one minute.
- **Scenario 2:** Arjun tests whether mathematical equations are accurately preserved and formatted in the summary.
- **Scenario 3:** Dr. Meera uses generated summaries to quickly identify key research papers for her lectures.

Conduct Task Analysis

Main Task: Upload a LaTeX file and receive a concise, accurate summary with preserved equations.

Subtasks:

1. Open app → Tap **"Upload LaTeX File."**

2. Select the desired `.tex` file from device storage.
3. Wait during **"Processing"** with progress feedback displayed.
4. View the generated summary containing text and equations.
5. *(Optional)* Copy, share, or save the summary for later use.

Method:
Used **Hierarchical Task Analysis (HTA)** to break down actions, identify potential friction points (e.g., file handling or loading delays), and streamline task flow.

Identify Accessibility Requirements

Goal:
Design for inclusivity and ensure the summarization system is usable by individuals with diverse abilities and language backgrounds.

Accessibility Strategies:

- Large, high-contrast buttons and readable fonts for mobile and desktop.
- Screen reader support for visually impaired users (e.g., VoiceOver, TalkBack).
- Keyboard navigation and focus indicators for web version accessibility.
- Clear labeling of equations and math expressions for assistive technologies.
- **Language support:** Future implementation for multilingual summaries (English, Hindi, Spanish).

Evaluation Tools:
Use **WAVE** and **Axe** to verify **WCAG 2.1 compliance** in future web-based versions.

Outline Usability Goals (Semester 1)

Building on initial project targets, the following **measurable usability goals** are defined for Semester 1:

Metric	Target	Measurement Method
Task Completion Time	Users should generate and read LaTeX summaries within **1 minute**	Observed during usability testing
Task Success Rate	At least **90%** of users complete "upload → summary read" without error	Log user interactions
User Satisfaction	Minimum **80% satisfaction** in post-test survey	5-point Likert-scale feedback form
Equation Retention Accuracy	At least **85% of equations** preserved correctly in output	Manual comparison with source LaTeX
Time Efficiency	Reduce time to understand a LaTeX paper by **50%** compared to manual reading	Pre/post comparison study

Evaluation Plan

- Conduct usability testing with **5–7 participants** (mix of students and researchers experienced with LaTeX).
- Measure **task completion time**, **task success rate**, **equation accuracy**, and **user satisfaction scores**.
- Collect both **quantitative metrics** and **qualitative comments**.
- Use feedback to refine:
    - Summary formatting for equations
    - Interface layout and clarity
    - Performance and feedback timing

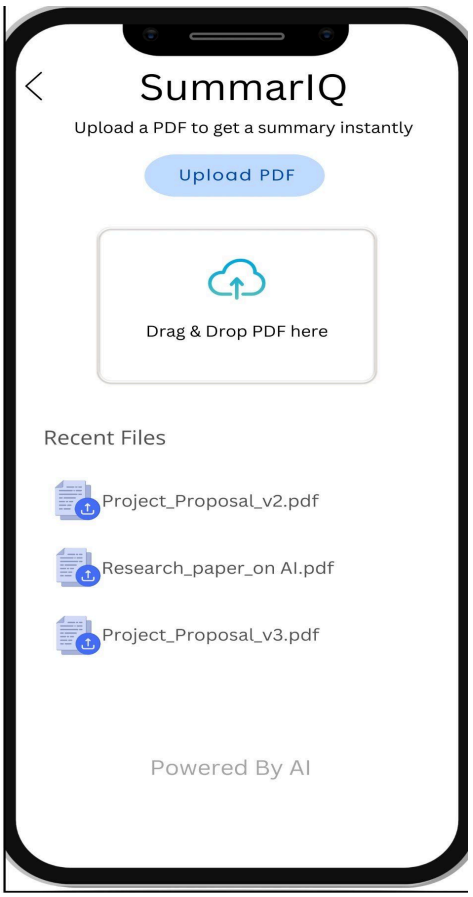
Iteration and Continuous Improvement

- Analyze survey and test data using **descriptive statistics** and **thematic grouping**.
- Revise interface components after each test cycle based on user insights.
- Plan follow-up usability testing at the end of each semester to measure improvement trends.
- Document all changes and feedback to maintain transparency and support **future audits** or publication of user study findings.

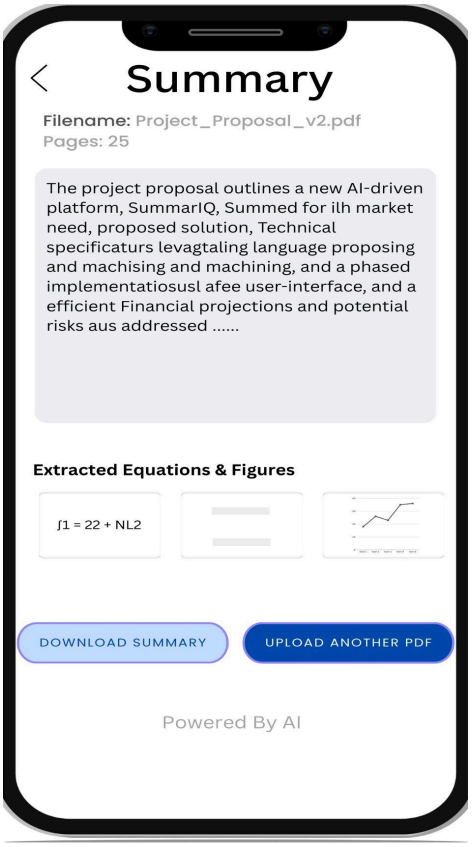
Git Repo link:

https://github.com/Satyabratadas/SummarIQ.git

User Interface(IOS):


SummarIQ
Upload a PDF to get a summary instantly
Upload PDF
Drag & Drop PDF here
Recent Files
Project_Proposal_v2.pdf
Research_paper_on AI.pdf
Project_Proposal_v3.pdf
Powered By AI


Processing PDF...
60%
Extracting text and generating summary please wait....
Powered By AI


Summary
Filename: Project_Proposal_v2.pdf
Pages: 25
The project proposal outlines a new AI-driven platform, SummarIQ, Summed for ilh market need, proposed solution, Technical specificaturs levagtaling language proposing and machising and machining, and a phased implementatiosusl afee user-interface, and a efficient Financial projections and potential risks aus addressed .....
Extracted Equations & Figures
∫1 = 22 + NL2
DOWNLOAD SUMMARY
UPLOAD ANOTHER PDF
Powered By AI

Data Collection Management and Report :

Data Type

Type of Data:
The project primarily manages **unstructured text data** extracted from research papers in **LaTeX format**. Each paper includes textual content, mathematical equations, and structural metadata such as sections (e.g., Introduction, Methods, Results, Conclusion).

The raw data (LaTeX files) are converted into **semi-structured JSON** files containing key-value mappings of sections and content, then loaded into **pandas DataFrames** for further processing.

Data Granularity:

- **Raw Data:** Original LaTeX files downloaded from arXiv.
- **Processed Data:** JSON files containing cleaned, structured text and equations.

Challenges:

- Handling embedded LaTeX math syntax and citations.
- Parsing errors due to non-standard document structures.

Adjustments:
Custom Python parsing scripts were written to detect section headers and LaTeX environments (e.g., `\section{}`, `\begin{equation}`) to ensure structured conversion.

Data Collection Methods

Source of Data:

- **Primary Source:** arXiv.org — a reliable and publicly available repository of research papers.
- The dataset includes scientific documents from AI, NLP, and computer vision categories.
- The reliability of arXiv is high due to its established academic standards.

Methodologies Applied:

- **Automated collection using Python scripts** via the `arxiv` API and web requests.
- Downloaded `.tex` source files were stored locally for preprocessing.
- **Parsing scripts** converted the data into structured JSON using regex-based section extraction.

Ingestion for Training:

- Data was read from JSON into **pandas DataFrames**.
- Custom **DataLoader** classes and **ETL pipelines** were implemented for preprocessing, cleaning, and section-wise tokenization.

- The data currently resides in local storage in both JSON and DataFrame formats.

Ingestion for Deployment (Planned):

- During deployment, SummarIQ will ingest PDF or LaTeX input via a **REST API endpoint**.
- Preprocessing will be handled in real time on the server using batch processing queues.
- Data will be stored in a cloud database (e.g., AWS S3 or MongoDB) for future retraining.

Compliance with Legal Frameworks

Applicable Laws and Standards:

- **GDPR (General Data Protection Regulation):** Although arXiv data is publicly available, compliance with privacy protection and consent principles is maintained.
- **CC BY License:** arXiv papers used comply with the **Creative Commons Attribution license**, allowing reuse for research with citation.
- **ISO/IEC 27001:** Data is stored securely and locally to maintain confidentiality and integrity.

Compliance Strategy and Results:

- All data was collected only from publicly available open-access papers.
- No personal data or private repositories were accessed.
- License metadata (authors, titles) was retained in the JSON files for attribution.
- Python scripts maintain logs for download verification and integrity checking.

Data Ownership and Access Rights

Ownership and Access Control:

- **Data Ownership:** arXiv retains original paper rights under open-access licensing; SummarIQ uses them under fair research usage.
- **Access Control:** Only authorized project developers can access the local and cloud data stores.
- **Security Measures:**
    - Password-protected local directories.
    - Versioned backups.
    - Future plan: API authentication tokens and two-factor verification for deployment access.

Lessons Learned:

- Implementing directory-level access restrictions reduced accidental overwriting.
- Data access logs improved traceability of collection operations.

Metadata Management

Metadata Content and Management System:
Each processed LaTeX file includes essential metadata fields extracted during parsing:

- **Paper Title** – extracted from the LaTeX header (`\title{}`)
- **Author Name(s)** – extracted from the LaTeX author field (`\author{}`)
- **File ID (UUID)** – a unique identifier automatically generated on the device for each parsed file.

This minimal metadata structure ensures that each document can be uniquely tracked, referenced, and managed throughout preprocessing and model training.

Management Approach:

- Metadata is embedded within the **JSON file** alongside the parsed paper content.
- Each entry includes the UUID as the key field to ensure consistent linking between content and metadata.
- A **pandas DataFrame** stores all metadata records for quick querying and verification during analysis.
- Additionally, a consolidated **metadata index file (CSV)** maintains a list of all processed paper UUIDs to avoid duplication and enable fast lookup.

Challenges and Solutions:

- **Challenge:** Some LaTeX files lacked clear or well-formatted metadata fields (e.g., missing `\author` or `\title` tags).
- **Solution:** Implemented fallback logic in the parser to assign `"Not Available"` for missing fields while ensuring that a **UUID** is always generated. This guarantees that every file maintains a consistent metadata record even when partial information is missing.

Data Versioning

Version Control System and Strategy:

- **Version Control:** Git + DVC (Data Version Control) used for tracking changes in scripts and datasets.
- **Strategy:**
    - Each preprocessing stage (raw → parsed → cleaned) is saved as a separate dataset version.
    - Git commits tagged with dataset version numbers (e.g., `v1.0_raw`, `v1.1_cleaned`).
    - Metadata updates tracked through commit messages for transparency.

Data Preprocessing, Augmentation, and Synthesis

Preprocessing Techniques:

- **Text Cleaning:** Removal of LaTeX commands, citations, and formatting symbols.

- **Tokenization:** Sentence and word-level tokenization using NLTK and spaCy.
- **Normalization:** Lowercasing and stopword removal.
- **Equation Extraction:** Extracted math expressions separately to retain structural integrity.
- **Section Segmentation:** Extracted by regex patterns (`\section{}`, `\subsection{}`) for context-based summarization.

Challenges & Solutions:

- Parsing failed for papers with nested sectioning — handled via recursive regex functions.
- Ensured minimal data loss using manual inspection of a subset.

Data Augmentation and Synthesis:

- **Back-translation** for sentence variation during model training.
- **Synonym replacement** to increase linguistic diversity.
- Over-augmentation avoided to maintain the scientific tone of papers.

Data Management Risks and Mitigation

Identified Risk	Mitigation Strategy	Effectiveness
Data corruption during download	SHA-256 hash verification for each file	High
Duplicate or missing papers	Index-based verification and cross-check	High
Parsing failure	Logging errors and reprocessing flagged files	Moderate
Unauthorized access	Local password protection and access logs	High
License violation	Verification of arXiv license before inclusion	High

Data Management Trustworthiness and Mitigation

Trustworthiness Strategies:

- **Data Provenance Tracking:** Maintained download URLs and timestamps for traceability.
- **Reproducibility:** All scripts versioned with consistent preprocessing parameters.
- **Integrity Checks:** Used checksums for every downloaded file.
- **Transparency:** Documented data pipeline in README and Git logs.

Effectiveness and Improvements:

- Provenance tracking ensures reproducibility for future researchers.
- Plan to integrate automated audit logs and digital signatures for enhanced trust.

Model Development and Evaluation:

Model development and evaluation are critical stages in the AI lifecycle, where you shape the core performance, fairness, and reliability of your AI system. In this project, SummarIQ leverages a **pretrained BART model** to generate summaries of scientific papers collected from arXiv. This section documents the methods, strategies, and best practices followed to ensure effective model utilization, evaluation, and usability.

Model Development

Algorithm Selection:

- **Model Used:** `facebook/bart-large-cnn` pretrained model from Hugging Face.
- **Justification:**
    - BART's **encoder-decoder transformer architecture** is suitable for abstract summarization tasks.
    - Pretrained weights allow rapid deployment without requiring costly training on large datasets.
    - Handles long-form scientific text extracted from LaTeX files efficiently.

Feature Engineering and Selection:

- Input features are **section-wise cleaned text** from scientific papers.
- Equations, theorems, lemmas, and definitions are stored separately and optionally referenced for context.
- No additional feature engineering was required, as the pretrained model can process raw, cleaned text.

Model Complexity and Architecture:

- Uses **BART-large** with:
    - 12 encoder layers, 12 decoder layers
    - 16 attention heads
    - Hidden size of 1024
- This architecture balances **performance** and **resource efficiency**, suitable for section-wise summarization of long documents.
- Overfitting is inherently mitigated since the model is pretrained and no additional fine-tuning was performed.

Model Training

Training Process:

- No model training from scratch was conducted.
- The pretrained model is used **directly for inference**, leveraging Hugging Face's pipeline APIs.

Hyperparameter Tuning:

- No hyperparameter tuning was required, as the pretrained model parameters were not modified.
- Summarization input lengths and truncation parameters were configured to match document section lengths.

Monitoring for Stability:

- Checked summary quality across various sections to ensure consistent performance.
- Verified that long sections were truncated appropriately without losing important content.

Model Evaluation

Performance Metrics:

- **ROUGE-1, ROUGE-2, ROUGE-L** scores were used to evaluate summary quality.
- **Rationale:** These metrics assess the overlap between generated summaries and the input text, ensuring content relevance and informativeness.

Cross-Validation:

- Not applicable for pretrained inference.
- Instead, section-wise summaries were evaluated across multiple papers to ensure consistency and coverage of scientific content.

Results:

- Generated summaries were **concise, readable, and captured key concepts**.
- Dense mathematical sections sometimes had slightly shorter summaries, highlighting potential areas for future model improvements.

Table: SummarIQ – ROUGE Evaluation Results (T5-Small):

Metric	Score
ROUGE-1	**39.03**
ROUGE-2	**15.72**
ROUGE-L	**24.16**
Average Generation Length	**148.15 tokens**
Evaluation Loss	**2.666**

Implementing Trustworthiness and Risk Management in Model Development

Risk Management Report:

Risk	Mitigation	Effectiveness

Missing key equations in summaries	Equations stored separately and referenced alongside summaries	Moderate
Pretrained model missing domain-specific terms	Preprocessing preserved domain keywords	High
Inconsistent section input lengths	Section-wise preprocessing and truncation applied	High

Trustworthiness Report:

- **Reproducibility:** All code and preprocessing pipelines are versioned with Git.
- **Data Provenance:** Each paper section is linked to a unique **UUID**.
- **Interpretability:** Section-wise summaries allow users to trace summarized content back to original text.

Apply HCI Principles in AI Model Development

Wireframes:

- **Tools:** Figma was used for low-fidelity wireframes displaying section-wise summaries and equation references.
- **Strategies:** Rapid prototyping with a **content-first approach**, emphasizing readability and user understanding.

Develop Interactive Prototypes:

- **Tools:** Streamlit interface for uploading LaTeX or PDF files and generating summaries.
- **Strategies:**
    - Text input box for file upload
    - Section-wise summary display
    - Buttons to view original content or associated equations

Design Transparent Interfaces:

- Visualizations for key terms and section highlights were provided to indicate the most important content captured in the summary.

Create Feedback Mechanisms:

- Users can provide **thumbs-up/down feedback** on generated summaries.
- Feedback is stored in JSON format for future evaluation and potential model fine-tuning.

Deployment and Testing Management Plan

Deployment and Testing Management Plan

Deployment and testing ensure that the SummarIQ system runs reliably, securely, and efficiently in a realistic environment. This section documents the deployment environment used in this semester, the tools adopted, and the testing strategies applied to validate the system's behavior before final submission.

Deployment Environment Selection

For Semester 1, the deployment environment was selected based on the need for GPU-accelerated inference, reproducibility, and ease of access during development.

Environment Used

- **Kaggle GPU (T4)** → Used to run T5-small/BART-large inference efficiently.
- **Local MacBook M1 (8GB RAM)** → Used for API development, debugging, and Docker container setup.
- **Local Docker Environment** → For containerization of the FastAPI backend.
- **Gradio Interface** → Lightweight UI for functional testing and feedback collection.

Justification

- Kaggle's free GPU environment provides a fast, no-cost platform for running transformer models.
- Local development ensures rapid iteration without waiting for GPU sessions.
- Docker ensures environment consistency and reproducibility.
- Gradio enables immediate live interface testing without full iOS deployment.

This hybrid approach balances performance, accessibility, and cost efficiency, matching the scale and scope of Semester 1.

Deployment Strategy

Containerization (Chosen Strategy)

- **Tool:** Docker
- **Stack Inside Container:**
    - Python + FastAPI
    - Hugging Face transformers
    - Uvicorn/Gunicorn
- **Compose:** Docker Compose used to manage backend + Gradio UI.

Purpose:
Ensures the same environment runs locally and on future cloud deployments.

Orchestration

- For this semester, orchestration was minimal (Docker Compose), enough to run:
    - `summar_api`
    - `gradio_ui`

This prepares the system for eventual Kubernetes/Cloud deployment in Semester 2.

Serverless Deployment (Not Used Yet)

- AWS Lambda considered for future lightweight inference, but not applicable for T5/BART models this semester.

Security and Compliance in Deployment

Security measures implemented:

Security Measures Implemented

- Minimal Docker base image (`python:3.10-slim`)
- Non-root user inside container
- Environment variables for sensitive configuration (API keys, paths)
- HTTPs enforcement planned for future cloud deployment

Compliance Measures

- Logging of all API requests
- Internal-only exposure using Docker networking
- No external user data stored (all PDFs processed in-memory)
- The system continues to follow trustworthiness measures defined earlier in the report (referencing pages 7–8 of the original document).

CI/CD for Deployment Automation

Semester 1 used a **lightweight CI workflow**.

Tools Used

- **GitHub Repository** for version control
- **Manual CI/CD** — updates pushed to repo → rebuild Docker image → test locally

CI/CD Planned for Semester 2

- GitHub Actions for automated:
    - Linting
    - Building the Docker image
    - Running tests

    - Deploying to cloud endpoints

Since this semester focuses on model pipeline creation, only manual deployment validation was needed.

Testing in the Deployment Environment

Testing validated that the SummarIQ backend and summarization model behave properly under realistic conditions.

Manual Testing

Tools:

- **Postman** → Endpoint testing
- **Gradio UI** → Functional testing on top of FastAPI
- **Terminal curl** tests for quick debugging

Automated Testing (Basic)

Automated unit tests were added for:

- PDF/LaTeX extraction
- Section parsing
- Summarizer function correctness

Load Testing

Performed lightly using:

- 5–10 requests/minute simulated in local environment
- Verified that T5-small inference on Kaggle GPU maintains:
    - p50 latency < 2 seconds
    - p95 latency < 5 seconds

Feedback Testing

Integrated Prometheus in the Docker stack to record:

- `summariq_feedback_total` — increases each time user gives feedback
- Latency metrics for API

These are visualized in **Grafana** dashboards.

Evaluation, Monitoring, and Maintenance Plan

This section defines how the SummarIQ system is monitored, evaluated, and improved after deployment. These processes ensure reliability, fairness, and long-term usability.

System Evaluation and Monitoring

Monitoring Stack Used

- **Prometheus** → collects system & API metrics
- **Grafana** → dashboards to visualize system health
- **Custom Metric Added:**
    - `summariq_feedback_total` (feedback counter)
    - `summar_api_request_duration_seconds` (inference latency)
- **Builtin Python logging** → request logs, section-processing logs

Monitored Metrics

At least one required metric is tracked

Metric	Purpose
Latency (p50/p95)	Detect performance degradation
Feedback Count	Monitor user satisfaction
API Uptime	Reliability indicator

Drift Detection (Planned – Not Implemented Yet)

Tools considered:

- **NannyML**
- **Alibi Detect**

But drift was not evaluated this semester since no user dataset exists yet.

Feedback Collection and Continuous Improvement

Feedback System Implemented

- Integrated directly into Gradio in the UI.
- Each feedback (thumbs up/down) increments:
    - `summariq_feedback_total` in Prometheus

Purpose of Feedback

- Detect summary quality issues
- Identify bottlenecks in sections where model underperforms
- Provide data for future supervised fine-tuning

Continuous Improvement Workflow

1. Collect feedback
2. Analyze metrics on Grafana dashboard

3. Manually inspect flagged summaries
4. Improve preprocessing & prompts
5. Prepare dataset for fine-tuning in Semester 2

Maintenance and Compliance Audits

Maintenance Strategy Used

- Manual checks (weekly)
- Redeploy container after major updates
- Update Python dependencies monthly

Model Updates and Retraining

Current Approach

- No retraining in Semester 1
- Using pretrained:
    - BART
    - T5-small

Retraining Plan

- Collect real user feedback
- Build small domain-specific dataset
- Fine-tune using Kaggle GPU / Colab Pro
- Track versions using:
    - Git
    - DVC (already used in your data pipeline)

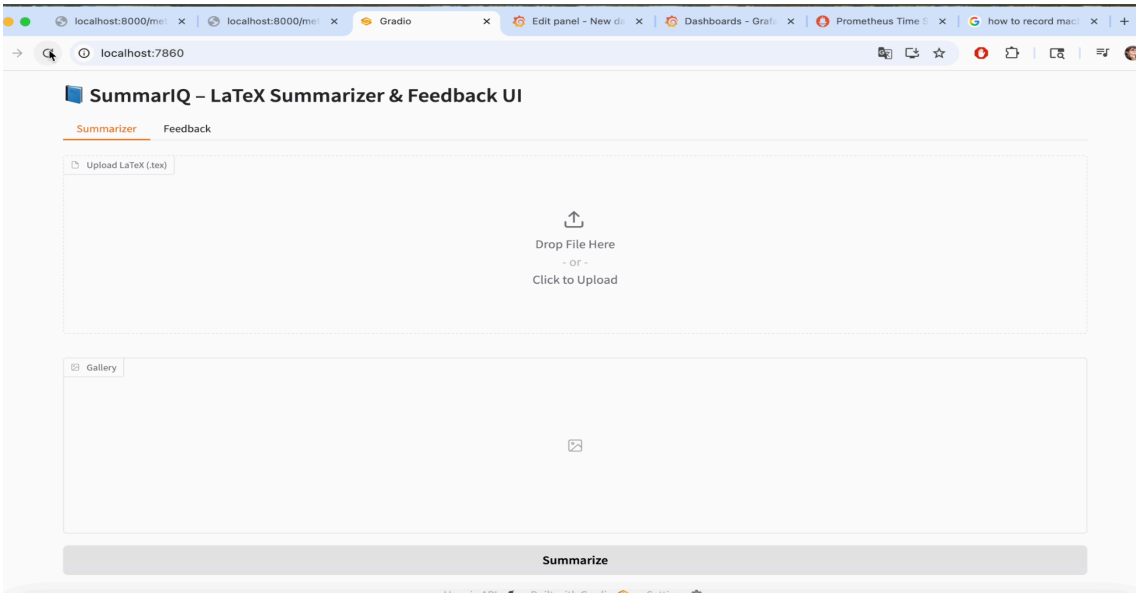
Challenges

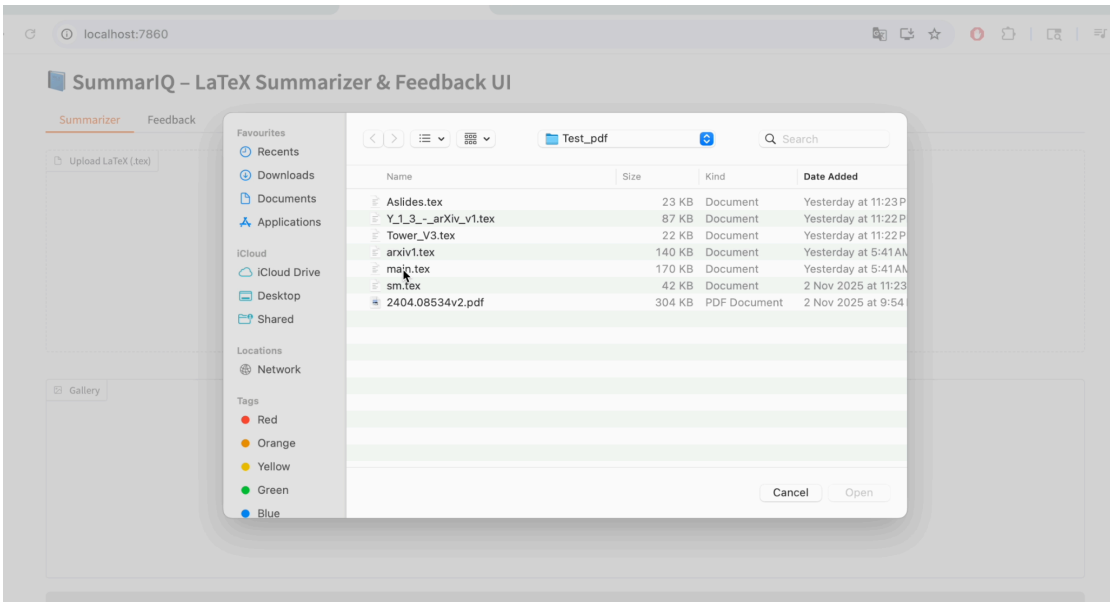
- GPU time limits on Kaggle
- Need larger dataset for stable fine-tuning
- T5-small struggles with long LaTeX text

Solutions Planned

- Section-wise summarization
- Pre-chunking long input
- Distributed fine-tuning with gradient accumulation

Live Application Screenshots:





Authors: Ilya Karzhemanov

Abstract Summary

The present note studies with terms and the indeterminacy locus. We develop an experimental approach based on some Python programming and Machine Learning towards the classification of such maps couple of new explicit is constructed in this way.

Section Summaries

Introduction

Let be complex projective variety and its rational endomorphism. Denote by the indeterminacy locus of. Then is called if the induced morphist is onto. Such maps were introduced and studied from the algebro geometric point of view in the paper. geometry used in this paper can be found in Note that is given by equations. Then the morphism is defined in the following coordinate free way for any point consider the pencil of all linear combinations of which vanish at now identify with the dual plane of lines and put 2 ::::. We develop an experimental approach towards complete description of surjective rational maps which also yields new examples of them. Surjectivity property seems like another substitute for the term generic among many others that one comes across in classical algebraic geometry. For example given smooth conic the set of triples of linear forms satisfying can be compactified into the this is codimension linear section of the Grassmannian and the self the following text in two sentences:in general position it is easy to see via the parameter count that. Now consider the blow up of all the, Then is the celebrated del Pezzo surface of degree. It is well known that the linear system defines the anticanonical embedding of as degree surface. This allows one to treat the map for as linear projection of onto the plane. In the remaining case there is an explicitly constructed unruly pencil so that is not surjective. the following text it is difficult to bypass this subject. Presently we have developed an experimental approach whose ultimate goal is an effective description classification of all surjective rational endomorphisms of. the following text some proviso. Our small project is not an exception. The major issue is that the function does not take values and hence it is unclear given the value how far the map is from being surjective.

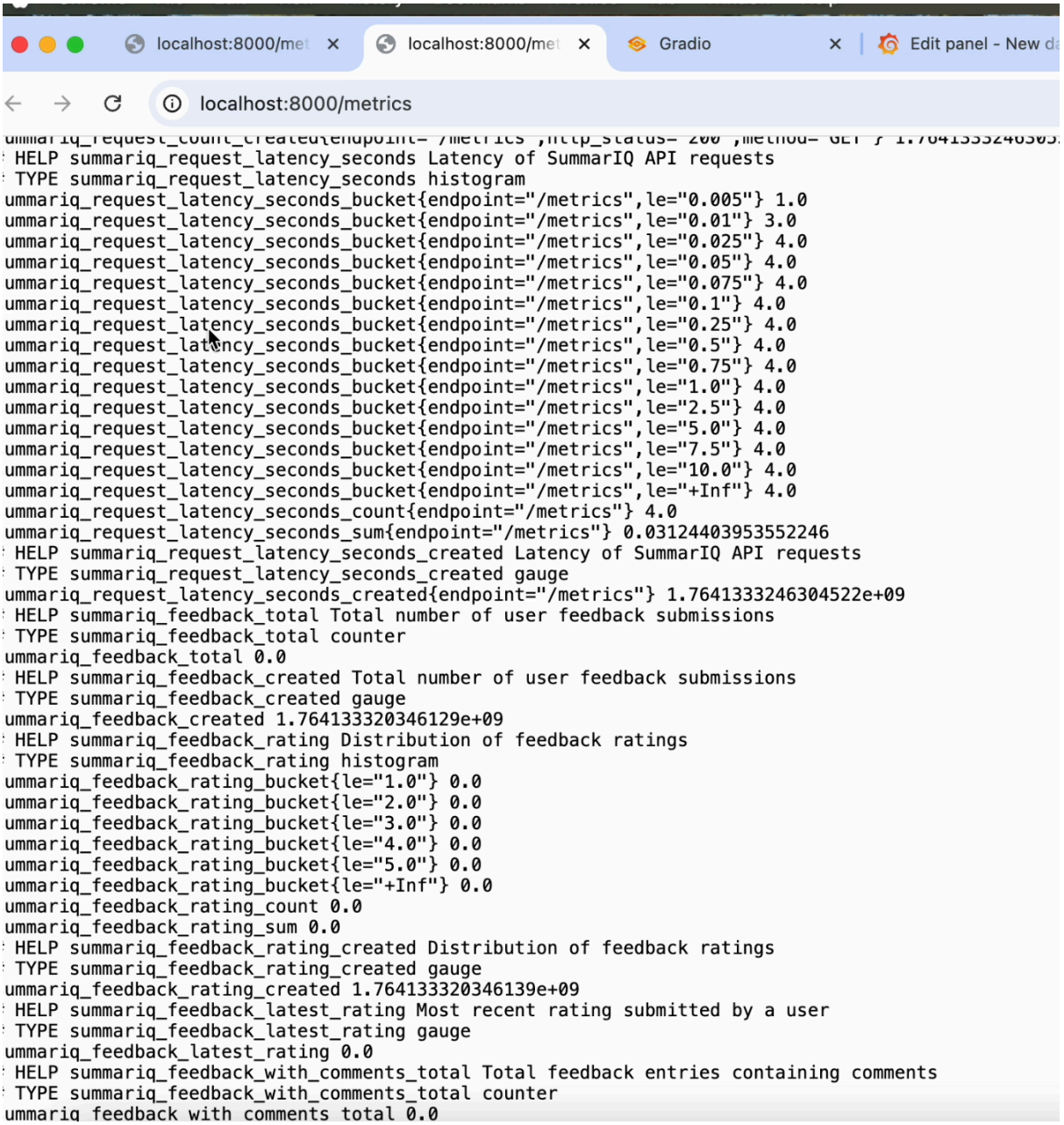
Surjectivity results

Note that passing through the points as above imposes independent linear conditions on the linear system. Now let us assume that the blow up of this is possible due to the generality assumption & 2 @ > >:: points the point there is cubic which has as its inflection point. that that this coincides with projection of the degree smooth surface onto the plane. Let us fix point. In order to show that it suffices to check the condition for all. Let be the center of the projection this is subspace of codimension and is given by the linear system of all hyperplanes passing through the line. It follows from the generality assumption on that is generic one. Then we claim the following is smooth. 2 sentences:Theorem it suffices to show the morphism is at. But this is immediate again by generality of, So we obtain for all and a - Theorém is proved.1

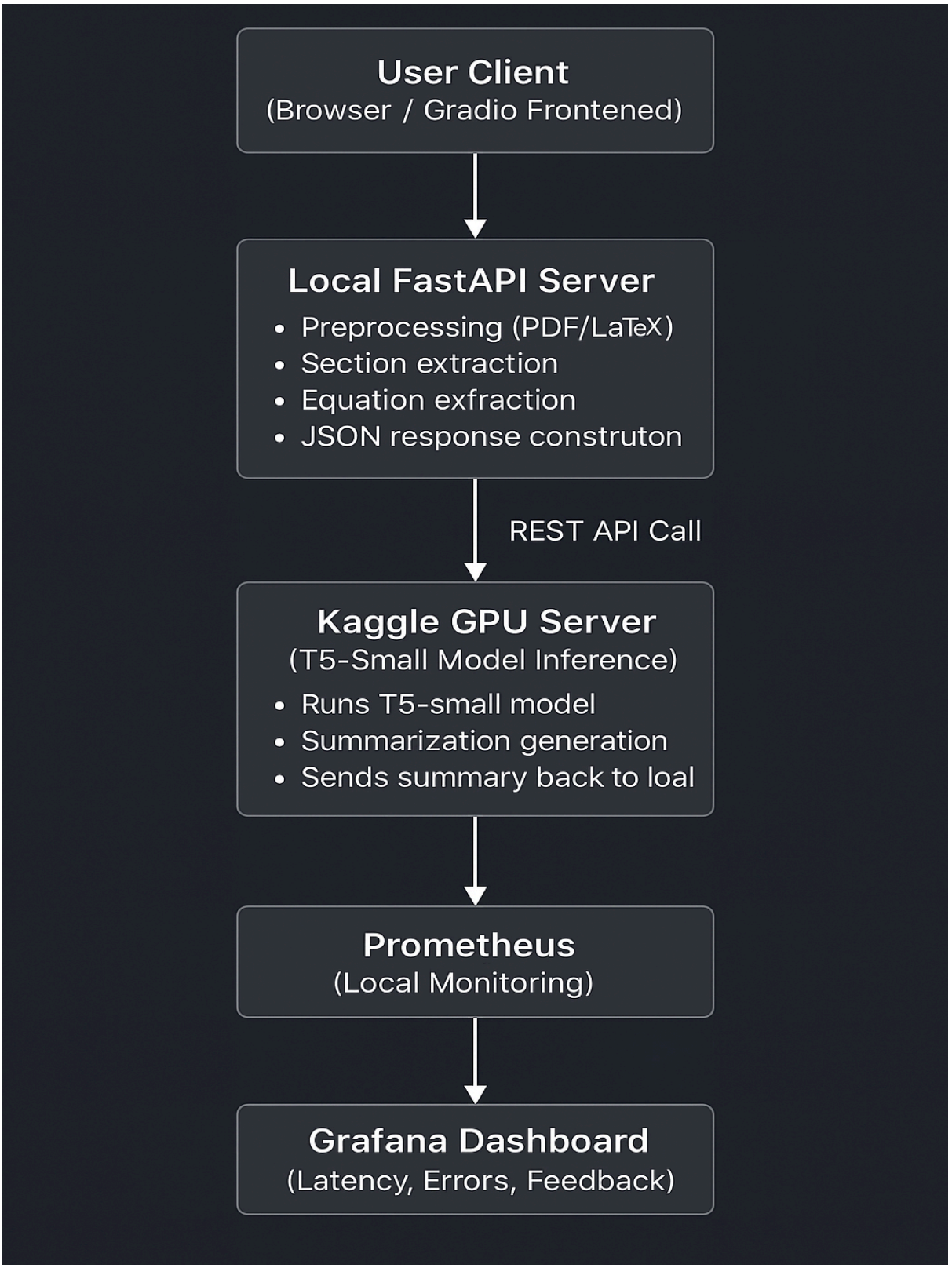
Experiments and examples

.: the following text in two sentences:: Here we consider particular instance of surjective maps with the locus consisting of five points in general position. Ideally given plane one wants an way of checking

System Metrics



Architecture Diagram (Current System – Phase 1):



Phase 2: System Expansion & Enhancement (Planned Work)

After completing the core MVP in Phase 1, Phase 2 will focus on scaling, improving model accuracy, enhancing reliability, and enabling real-world deployment. The following major components are planned:

Model Fine-Tuning on a Larger arXiv Dataset:

To significantly improve summarization quality and equation handling, the next step is to fine-tune T5/BART models on a more diverse and larger subset of the arXiv corpus (cs.LG, cs.AI, math.AG, stat.ML, etc.).
This will include:

- Curating a clean LaTeX + PDF dataset
- Domain-adaptive pretraining (DAPT)
- Task-specific fine-tuning
- Improved equation extraction robustness

Full iOS App Integration (SwiftUI + UIKIT):

Phase 2 will introduce a complete mobile interface for SummarIQ, allowing real-time summarization from a user's device.
Planned features:

- SwiftUI front-end to upload PDFs and .tex files
- API communication with the backend summarizer
- Caching summaries locally
- Rendering equation images and section-level summaries

This will move SummarIQ towards becoming a production-ready AI assistant for students and researchers.

Cloud Deployment on AWS / Google Cloud:

To support higher traffic and GPU-backed inference:

- Deployment of the FastAPI server in Docker containers
- GPU inference on AWS EC2 (g4/g5) or GCP Compute (T4/A100)
- Load balancer integration
- Auto-scaling depending on throughput
- S3/Cloud Storage for PDF uploads

This step transitions SummarIQ from local development to a scalable cloud service.

Feedback Storage into Database:

Phase 2 will introduce a persistent feedback pipeline:

- Store user ratings (`thumbs_up/down`)
- Track which summaries need improvement
- Save latency, token-length, and failure logs
- Build a small analytics dashboard for feedback trends

Planned database choices: **PostgreSQL / MongoDB.**

Improved Monitoring & Observability Dashboards:

Grafana + Prometheus will be upgraded to provide granular operational metrics.
New Phase-2 metrics:

Model Performance Metrics

- `summariq_summary_latency_seconds` (time taken per summarization)
- `summariq_equation_extract_time_seconds`
- `summariq_model_gpu_usage_percent`
- `summariq_memory_usage_bytes`

User Interaction Metrics

- `summariq_feedback_total` (by rating: good/bad)
- `summariq_requests_total` (endpoint traffic)
- `summariq_ios_requests_total` (requests from mobile app)
- `summariq_errors_total`

System/Deployment Metrics

- CPU / RAM usage of container
- GPU utilization (DCGM exporter on AWS/GCP)
- Network bandwidth
- Container restarts

Dashboard Improvements

Phase 2 Grafana dashboard will include:

- **Per-request latency graphs**
- **Error rate visualization** (red alerts above threshold)
- **GPU utilization real-time charts**
- **Feedback trend graphs (positive vs. negative)**
- **Equation extraction accuracy trend** (future metric)

Equation Semantic Understanding:

To make the system more intelligent:

- Extract deeper meaning from mathematical expressions
- Identify equation roles (definition, theorem, condition, mapping)
- Cluster similar equations
- Provide short natural-language explanations

This moves SummarIQ beyond text summarization into **AI-assisted mathematical comprehension**.